

Simulation of Quantum Computers: Review and Acceleration Opportunities

Modified copy for citation-provenance evaluation. Original: arXiv:2410.12660, CC-BY 4.0. The bibliography is the original's first 30 entries, unaltered. No reference or in-text marker has been changed. The untouched original is [eval/corpus/originals/2410.12660.pdf](https://arxiv.org/abs/2410.12660).

Simulation of Quantum Computers: Review and Acceleration Opportunities ALESSIO CICERO, Chalmers University of Technology and University of Gothenburg, Sweden MOHAMMAD ALI MALEKI, Chalmers University of Technology and University of Gothenburg, Sweden MUHAMMAD WAQAR AZHAR, ZeroPoint Technologies AB, Sweden ANTON FRISK KOCKUM, Chalmers University of Technology, Sweden PEDRO TRANCOSO, Chalmers University of Technology and University of Gothenburg, Sweden Quantum computing has the potential to revolutionize multiple fields by solving complex problems that can not be solved in reasonable time with current classical computers. Nevertheless, the development of quantum computers is still in its early stages and the available systems have still very limited resources. As such, currently, the most practical way to develop and test quantum algorithms is to use classical simulators of quantum computers. In addition, the development of new quantum computers and their components also depends on simulations. Given the characteristics of a quantum computer, their simulation is a very demanding application in terms of both computation and memory. As such, simulations do not scale well in current classical systems. Thus different optimization and approximation techniques need to be applied at different levels. This review provides an overview of the components of a quantum computer, the levels at which these components and the whole quantum computer can be simulated, and an in-depth analysis of different state-of-the-art acceleration approaches. Besides the optimizations that can be performed at the algorithmic level, this review presents the most promising hardware-aware optimizations and future directions that can be explored for improving the performance and scalability of the simulations. CCS Concepts:

- Hardware → Quantum computation;
 - Computing methodologies → Simulation types and techniques. Additional Key Words and Phrases: Quantum Computing, Computer Simulation, Hardware Acceleration, CPU, GPU, FPGA
- 1 Introduction As we try to solve more and more complex problems such as developing new chemical compounds or evaluating the physical properties of new materials [10], the demand for computational resources continues to grow. Certain problem cases are so complex that no existing computer system can solve them within reasonable time. For such cases, Quantum Computing is a promising emerging computing paradigm that can provide solutions to these problems [10, 20, 21]. As quantum computing is still in its early stages of development, real machines are scarce and the existing ones have limited compute resources. Although several quantum computing experiments have shown promising results and quantum computers are being scaled up to hundreds of qubits [4, 6, 15], full quantum advantage has yet to be achieved. In order for algorithms to be executed in a quantum computer, they are represented as quantum circuits. The ability of quantum computers to solve increasingly complex problems is limited by the maximum size of the executable quantum circuit. This size is mainly bounded by two factors: (1) the number of available qubits (the fundamental unit of a quantum circuit) and (2) the circuit Authors'ContactInformation:AlessioCicero,alessio.cicero@chalmers.se,ChalmersUniversityofTechnologyandUniversity of Gothenburg, Gothenburg, Sweden; Mohammad Ali Maleki, mohammad.ali.maleki@chalmers.se, Chalmers University of Technology and University of Gothenburg, Gothenburg, Sweden; Muhammad Waqar Azhar, waqar.azhar@zptcorp.com, ZeroPoint Technologies AB, Gothenburg, Sweden; Anton Frisk Kockum, anton.frisk.kockum@chalmers.se, Chalmers University of Technology, Gothenburg, Sweden; Pedro Trancoso, ppedro@chalmers.se, Chalmers University of Technology and University of Gothenburg, Gothenburg, Sweden. raM Jhp-tnauq[2v06621.0142:viXra 2 A. Cicero et al. depth, which refers to the number of distinct timesteps at which

quantum gates are applied. In current quantum computer implementations, not only are qubits few but also the time they are stable (i.e., their value remains reliable) is limited. This stable time is limited by the qubit implementation technology and also their sensitivity to fluctuations in the environment (e.g., magnetic fields and temperature). Quantum circuit execution is just one element of the Quantum Computing Stack [8]. Other elements include mapping an algorithm to a quantum circuit, generating pulses to execute gates on qubits, and reliably reading out the results. To develop and test the various parts of the quantum computing stack, accurate and fast simulation tools are essential. Similarly to classical computers, different simulation tools are needed at various design and verification steps, each with a specific focus and level of detail. A generic quantum computer system is composed of different parts, and the development of each part can be assisted by a different type of simulation. This review work focuses in particular on the simulation of the execution of quantum circuits. Simulations of quantum circuit execution can be performed at different levels, from simulating the behaviour of the quantum hardware platform, to simulating the interaction between a few qubits and a coupler forming a gate[27], and simulating entire circuits[12]. While several simulators have recently been deployed and made publicly available, the need to simulate more complex algorithms, circuit configurations, or more accurately model qubit behavior leads to an exponential increase in computational and memory demand for the simulations. Scaling up the simulations without exponentially increasing the amount of necessary resources, requires optimisations or approximations at different levels. As such, several classical computer techniques, such as data compression or optimized parallel execution, have been applied to reduce the memory requirements and accelerate the computations [11]. Several review works on classical simulation of quantum computers are available, but their focus is different from the acceleration of the simulation. Some works focus on the state-of-the-art numerical methods, such as the work by Xu et al. which gives a general overview, and Jones et al. which focuses on the full-state simulation techniques. Other works focus both on reporting the state of the art for simulators as well as giving a high-level overview of the acceleration approach as in the work by Young et al.. The work by Heng et al. focus on presenting optimisations for GPU execution while the review by Jamadagni et al. focuses on setting up a benchmark for multiple simulators available, and comparing their performance. In contrast to the previous works, our work addresses the topic in a different way, with a goal of focusing on the possible hardware-aware acceleration techniques. It provides up-to-date details about the available simulator approaches, tools and techniques, and focuses on the possible optimisations for a broader selection of hardware platforms. First, this review work presents an overview of the different types of simulators for the execution of quantum circuits. The simulators are organized into different categories, to help navigate the landscape of the available tools. Second, it provides an overview of acceleration techniques described in the state of the art to improve the performance of the simulations. Having a global overview of the existing proposed solutions, we will infer what the most promising trends are and speculate on the future directions for the acceleration of the simulation of quantum computers. In this context, different works propose different approaches to optimize the simulation on different hardware platforms, such as Central Processing Unit (CPU), Graphics Processing Unit (GPU), Field Programmable Gate Array (FPGA), or more complex setups such as a hybrid CPU and Quantum Processing Unit (QPU) core. A summary of the hardware-aware techniques observed in the analyzed works is presented and this is used as a basis to extrapolate what the future directions in hardware-aware acceleration for quantum computer simulation should be. The organization of this survey is as follows. Section 2 introduces the essential concepts in quantum computing necessary for understanding the rest of the sections. In Section 3 we give an overview of the various parts of a quantum computer. Section 4 covers simulations at different levels and provides an overview of the available simulators for each of them, also introducing the main bottlenecks for scaling up the number of simulated qubits. Section 5 details the most common hardware platforms used for simulation. The following sections cover different acceleration methods, sorted by platform: CPU in Section 6, GPU in Section 7, and FPGA in Section 8. A summary and indication of future directions for hardware-aware optimization is presented in Section 9, and Section 10 concludes this work.

2 Background

This section presents a quick overview and introduction to the basic quantum computer concepts.

representing the parallel gates will be $2^n \times 2^n$. If there are n successive gates, there will be the need for $n \times 2^n \times 2^n$ matrices to represent the quantum circuit.

2.4 Tensor networks Some of the simulation methods are based on the representation and execution of the quantum circuit as a tensor network. For example, a vector v or a matrix M : $v = (v_1, v_2, \dots, v_n)$ or $M = (M_{ij})$ are considered an order-1 tensor and an order-2 tensor, respectively while an order-3 tensor T is represented as shown in Figure 4(a). A tensor network is a high-dimensional tensor that may be viewed as a graph where each node represents a tensor and the edges represent the connections between them. A tensor of order- n is an object with n indices and can be represented with n legs [9], as shown in Figure 4(b). D E Text Contraction F i j k i k Fig. 5. The tensors D and E are contracted into a single tensor F, with indices i and k. In order to work with tensors, the main operation is the tensor contraction, shown in Figure 5. Contracting two tensors means performing a summation over an index, which contracts the internal indices between the two tensors, mathematically represented as: $\sum_i D_{ij} E_{ik} = F_{jk}$ (10). The usual approach to handle the tensor networks is to contract the network from order n to a single tensor. When the network contraction is divided in a sequence of binary contractions, the total computational cost is influenced by the choice of the contraction sequence. Working with pairwise contractions typically enables optimal computational performance, as they can be implemented as matrix-matrix multiplication.

3 Quantum computers Since the beginning of quantum research various algorithms have been explored to verify if a quantum computer could solve some problems faster than classical machines. Examples of such quantum algorithms are Shor's algorithm, Grover's search algorithm, and quantum simulation. Shor's algorithm, based on the quantum Fourier transform, can be used to find the prime factors of an integer. Grover's search algorithm allows to speed up the search for an element which satisfies a certain known property. It allows in the case of a search space of size N to find an element with no prior knowledge about the structure of the information in $O(\sqrt{N})$ operations instead of the $O(N)$ operations required classically. Quantum simulations are simulations of naturally occurring quantum mechanical systems, such as molecules, using quantum computers. Quantum computers utilize different quantum chip technologies, including ion trap, neutral atom [15], semiconductor, and photonic. This work focuses on the currently most common technology used to build quantum computers: the superconducting architecture. Figure 6 shows the generic structure of a superconducting quantum computer.

3.1 Quantum chip The quantum chip, which includes the physical qubits, is the main component of the quantum computer. Depending on the technology, they are controlled and connected in different ways. Connecting multiple qubits together allows for more complex circuits, comprising single- and multi-qubit gates. As already mentioned in the introduction, one of the main limitations of current physical qubits is their short coherence time. This is the timescale of the exponential decay of the qubit superposition state [3]. The correctness of quantum computers is usually measured in terms of the fidelity of the operations or the output state. Coherence times vary depending on the technology and the "quality" of the qubit. A different metric that allows to compare different technologies and different approaches is the amount of gates that can be executed during the coherence time of a single qubit. In the case of the current Readout ADC Host Quantum Chip Pulse DAC Generation Fig. 6. Basic scheme for a superconducting quantum computer and its control electronics. The control pulses generated are converted from digital to analog through the Digital-to-Analog Converter (DAC). The analog output of the quantum chip goes through an Analog-to-Digital Converter (ADC) and is then read. superconducting qubit implementations, they have a coherence time ranging from hundreds of microseconds[17], up to close to a millisecond. Consequently, the control system, responsible for providing pulses to activate the qubits and read out their state, must be fast enough to be able to control thousands of gates during this time. Although some technologies, such as photonics, allow the quantum chips to work at standard room temperature, most of the other approaches require the chips to be cooled down to a temperature close to absolute zero to avoid thermal noise. Therefore, the chips are usually are

placed inside cryogenic chambers. This introduces additional challenges related to controlling and communicating with the chips from other components of the quantum computer. If placed inside, the components need to operate at the low temperatures of the chamber and are forced to dissipate minimal power, while if placed outside the chamber, they face a bottleneck in scaling due to the physical limit of input and output cables available. In addition to the physical number of cables, each cable also carries heat into the chamber, thus increasing the thermal noise.

3.2 Pulse generation The qubit driving mechanism may vary depending on the type of quantum computers we briefly discussed in the previous section. In the case of the superconducting architecture, qubits are controlled by radio-frequency signals. Each qubit is characterised by a slightly different resonance frequency, which allows to drive them individually. The specific resonance frequency can vary slightly in time due to interaction with impurities in the qubit environment, affecting the fidelity of operations on each qubit. Therefore, periodic recalibration of the system is necessary [18]. This recalibration process allows to update the correct resonance frequencies used to control the qubit by the pulse generation hardware. Scaling up the number of qubits requires additional inputs to the system, which, as previously mentioned, becomes a major challenge for technologies that require a cryogenic chamber for cooling.

3.3 Readout After the execution of the quantum circuit, the value of all or some output qubit must be measured. The main challenge is that discrimination between zeros and ones is non-trivial, leading to the development of various readout techniques [7, 23]. The approaches range from improved amplification of the output at the analog level [7] to the introduction of error correction for fault-tolerant quantum computing using higher-abstraction level techniques such as machine learning. The main issue, as in the case of the pulse generation, is related to managing the connection between the readout system and the quantum chip through the cryogenic chamber, which is one of the major bottlenecks in scaling up the system. 8 A. Cicero et al.

3.4 Compiler Mapping an algorithm to a quantum circuit is a non-trivial task. To do this, we use specific tools such as a compiler, which must take multiple decisions. Ideally, the compiler would assign each qubit in the algorithm to a physical qubit in the quantum computer, without considering the different error probabilities of each qubit. In this case, all the multi-qubit gate operations would occur between neighbouring physical qubits, giving the compiler freedom to choose any quantum gate operation. In a real quantum computer, this is not always possible. First, different qubits on the same chip could have different probabilities of error due to the influence of the external environment. When reading the output of a qubit, the output might not always be correct, due to factors such as drifts in calibration, temperature variations, or measurement errors. Therefore mapping the operations to qubits according to their probability of error might increase the fidelity. Additionally, in some architectures, such as superconducting circuits, qubits can only interact with neighbouring qubits, requiring the addition of SWAP operations when direct interaction is not possible. Not every gate might be available on every quantum computer, and the choice of gate can influence the output fidelity. Multiple approaches [25, 30] have been proposed to address this issue, aiming to optimize the mapping of quantum circuit to the available qubits and gates. Thus, it is very important that the compiler considers all the characteristics of the available system as a sub-optimal compilation phase may result in resource under-utilization, high error rate, and lower fidelity.

4 Simulations of quantum computers In this section, we focus on how we can simulate, and optimize the simulation of, the execution of quantum computers. Simulations are becoming an increasingly valuable tool for evaluating and developing various components of the quantum computer stack. As in classical computing, each part of the system requires a distinct approach and level of detail.

Device Level Simulation Gate Level Schrödinger of quantum approach computers Algorithmic Level Tensor Approach Fig. 7. Taxonomy of simulation of quantum computers. Simulations of quantum computers can be classified into the following three main categories, as also shown in Figure 7.

- **Device level** — this is the lowest level of the system and focuses on the materials and the implementation of the qubit. This is comparable to the low-level classical hardware simulation.

- Gate level — this is the middle level, where gates are mapped to qubits. This is comparable to the micro-architecture-level classical simulation.
- Algorithmic level — this is the highest level of the system, where algorithms are mapped to quantum circuits. This is comparable to the functional-level classical simulation. Although it is possible to simulate a complete quantum computer for a small number of qubits, this does not scale well for increasing number of qubits. This issue arises at every level and it is better described in each subsection. In general, the simulation execution is both memory- and compute-bound. With scalability in mind, simply trading one for the other is insufficient to achieve the desired performance, as beyond a certain number of qubits, one of the two limitations will become impossible to overcome. We should notify the reader that in this work, we will not focus on the acceleration of device-level or gate-level simulations, as there are already plenty of solutions. Instead, we will focus on the research field of algorithmic-level simulation.

4.1 Device level Device-level simulations help address the mechanical, thermal, and electromagnetic aspects of the problem. In the case of superconducting circuits, some mainstream commercial tools are reported here:

- Ansys Electronic Desktop solves 3D Maxwell equations, and eigen-equations with various boundary conditions, including the option for lumped-element boundary conditions. This is useful for nearly all problems in the electromagnetic domain at RF and microwave frequencies.
- COMSOL Multyphysics[2] is a Finite Elements Method (FEM) solver of many partial differential equations of physics. It is useful for simulating thermal, mechanical, and electromagnetic problems. London's equations can be solved to simulate the Meissner effect in packages, chips, resonators, as well as effects of kinetic inductance. Heating or cooling of circuits and the impact of mechanical strain on packages or Printed Circuit Boards(PCBs) can also be simulated.
- SolidWorks[29] can be used for designing of mechanical parts such as nuts and bolts, packages, fixtures, heat sinks, and connectors. Designs can be exported and re-imported in Ansys tools for electromagnetic simulations. It is also possible to export the FEM mesh for use in other FEM-based simulators. FreeCAD is an open-source (python-based) version of this tool.
- InductEX from SunMagnetics Inc. solves London's equations using FEM and is primarily used for extracting inductances in the layout of superconducting circuits, such as Rapid Single Flux Quantum (RSFQ) logic gates. Different types of simulators will be used in quantum dot technology, such as Intel Quantum Dot Simulator and nanoHUB Quantum Dot Lab. Although related to different technologies, the simulators can be classified at the same device level. They evaluate the selected material and geometry and calculate information such as 3D visualization of the confined wave functions, incident light angle and polarization, and isotropic optical proprieties.

4.2 Gate level Gate-level simulation focuses on the interaction between a few qubits connected by couplers, which are used to implement quantum gates. Evaluating different configurations is necessary to study and define the actual gates that will be used in quantum circuits. An example circuit used for gate-level simulation is shown in Figure 8. Simulations at this level are based on solving differential equations derived from the Schrödinger equation. There are different types of simulations at this level. Static gate-level simulation is useful for evaluating the energy levels of qubits and couplers, as well as assessing the interference between them. Dynamic gate-level simulation helps evaluate the effects of a pulse used to control the gate. 10 A. Cicero et al. g12 omega1 omega1 alpha1 alpha1 Fig. 8. Simple circuit used for gate-level simulations, composed of two qubits and one coupler to connect them. Each qubit is modeled as a Josephson junction and a capacitor. In the circuit simulations a capacitive coupling is used to connect the two qubits. This is crucial for determining the type and frequency of pulse needed on the quantum chip to ensure each qubit responds as expected. Examples of simulations at this level are CSQR and scQubits [27]. The gate-level simulation acceleration is based on the optimisation of the differential equation solution. Other simulators, such as QuTiP, allow for run-time optimisation of the type of solver and hardware configuration, enabling

faster simulations or reduced memory usage. Dynamiqs is another simulator that can run on CPU, GPU, and TPU, offering speedup through batching and parallelization on CPUs and GPUs. Another simulator at this level is QuantumOptics.jl, but no information on the acceleration is available.

4.3 Algorithmic level Algorithmic-level simulation is useful for testing the correct functioning of potential quantum algorithms. ψ H | $\square$ | $\square$ | $\square$ 0 H | $\square$ | $\square$ | $\square$ 0 H | $\square$ | $\square$ | $\square$ Fig. 9. 9-qubit Shor's code circuit for error correction. Different gates are used in this circuit: multiple Hadamard gates which change the qubit state to the superposition state, and CNOT gates. One such algorithm is Shor's 9-qubit error-correction code shown in Figure 9. Unlike device-level simulations, algorithmic-level simulations generally treat the quantum circuit as a sequence of matrix operations. Simulations may include noise, which models external factors such as temperature, electrical interference and additionally device changes in time, that may affect the state of the qubits. Noisy quantum states are represented as a density matrix, which requires more storage than the vector of a pure quantum state. Consequently, noiseless simulations need less memory and compute time, but they are farther from real-world conditions. Nowadays, we are moving towards Noisy Intermediate-Scale Quantum computing (NISQ) [14]. Introducing noise in the simulations can provide a better correlation between simulation and real-world results. Below is a list of some of the most common algorithmic-level quantum circuit simulation tools available for use:

- Intel Quantum SDK: Provides two different simulators with two different modes of simulation, Generic Qubits and Intel Hardware. The Generic Qubits simulation is a state- vector-based simulation, which uses the Intel Quantum Simulator as a backend, allowing for a qubit-agnostic execution. The Intel Hardware mode uses as its core the Quantum Dot Simulator, which simulates the Intel quantum hardware, currently under development.
- QuEST: Simulator using state vectors and density matrices for quantum circuits, which uses multithreading, GPU acceleration, and distribution to optimise the execution on multiple types of devices such as laptops, desktops, and networked supercomputers.
- Qiskit: Open-source SDK for working with quantum computers. Used as the core library for different types of simulations. It allows both for noiseless and exact noisy simulation with Qiskit Aer.
- HyQuas: This is a hybrid partitioner-based quantum circuit simulator, optimized for GPUs. It selects the optimal simulation method for different parts of a given quantum circuit. It implements additional solving methods and makes use of distributed simulation.
- qTask: Quantum circuit simulator that focuses on optimizing the speed in case of modifications of a small part of the circuit, defined as incremental simulation. When this happens, the state amplitudes (probability amplitudes of the state) are incrementally updated, removing the need for simulating the whole circuit every time.
- PennyLane [12]: Open-source software framework for quantum machine learning, quantum chemistry, and quantum computing. Supports GPU acceleration by making use of NVIDIA cuQuantum SDK [11].
- QuTip-qip: QuTiP quantum information processing package, which offers two ap- proaches to quantum circuit simulation. The algorithmic-level approach is based on the circuit evolution under quantum gates by matrix multiplication, while another approach uses open system solvers in QuTiP to simulate noisy quantum devices. While gate-level simulations focus on the full time evolution from the start to the end of the gate, algorithmic-level simulations consider only the changes that the gate's corresponding matrix applies to the state vector. Although these simulators provide less detail, they allow for faster simulations or alternatively an increased number of simulated qubits. However, they are also constrained by memory and time. The memory required to simulate a circuit is determined by the number of qubits, the number of gates, the data representation, and the number of times the circuit is executed, which, in the case of a real quantum circuit, corresponds to the number of measurements. 12 A. Cicero et al. Most simulators are designed for algorithmic-level simulation, which is the furthest from actual physical implementation and allows for

evaluating problems from a hardware-agnostic perspective. Quantum simulations at the algorithmic level can be categorized into three main types of approaches:

- Schrödinger-style simulation
- Feynman-style simulation [13]
- Tensor-based simulation While the Schrödinger and Feynman approaches have been the primary methods in the past, there is now greater focus on exploring the potential of the tensor-based approach.

4.

3.1 Schrödinger-style simulation. Schrödinger-style simulation, also known as state-vector-based simulation, is the mainstream technique for general-case simulation of quantum algorithms, circuits, and physical devices. As described earlier, a quantum state is represented by a vector of complex-valued amplitudes, and the primary approach in this simulation is to store the current state vector and iteratively multiply it by a state transformation matrix. Schrödinger-style simulation resources scale linearly as a function of the circuit depth and exponentially as a function of the number of qubits. Additionally, it allows for relatively straightforward implementations that are commonly used for small and mid-size quantum circuits and device/technology simulations.

4.

3.2 Feynman-style simulation. Feynman-style path summation considers each gate connecting two or more qubits in a quantum circuit as a decision point from which the simulation branches. The final quantum state is obtained by summing up the contributions of the results of each branch, which are calculated independently. In comparison to Schrödinger-style simulations, traditional Feynman-style path summation [1] uses very small amounts of memory but doubles the runtime on every (branching) gate. This results in a much longer runtime and does not allow for optimal memory usage, as the number of paths grows exponentially as a function of the decision points. Unlike traditional Schrödinger-style simulation, Feynman's resulting algorithms are depth-limited, making them a good fit for near-term quantum computers that rely on noisy gates.

4.

3.3 Schrödinger-Feynman hybrids. A possible approach to leverage both Schrödinger and Feynman approach is Schrödinger-Feynman hybrids [1][26], proposed in the work of Markov et al.. In the context of nearest-neighbour quantum architectures [6, 16], where the qubit array is partitioned into sub-arrays, the Schrödinger approach is applied to each sub-array. This reduces the memory requirements for n qubits from 2^n to $2^{n/2+1}$, although this introduces a dependency on the number of gates acting across the partition. The gates are decomposed into a sum of separate terms to allow for independent simulation, but this also results in an increase in execution time. For example, in the case of a CZ gate, the gate can be decomposed into two terms, which results in doubling the run time. However, this increase in run time is still lower compared to the Feynman-style path summation described earlier.

4.

3.4 Tensor-based simulation. The Schrödinger approach typically stores the full state of the qubits, allowing for the simulation of arbitrary circuits. However, the downside is the exponential increase in memory requirements. For an n -qubit system, 2^n space is needed. When using the tensor-based approach, the quantum circuit is described as a tensor network, with each n -qubit gate represented as a rank- $2n$ tensor. This transforms the simulation into a problem of contracting the corresponding tensor network. Tensor network contraction is performed by convolving the tensors until only one vertex remains. For circuits with a large number of qubits and shallow depth (as complexity often grows exponentially with circuit depth), this method is State vector Compression Data format optimisation Schrödinger Gate clustering Circuit partitioning CPU Tensor network creation optimisation Tensor-based Tensor network contraction

optimisation Data format optimisation Circuit partitioning Schrödinger Memory access optimisation Parallel execution optimisation Algorithmic Level Sim- GPU ulation Acceleration Tensor network cre- ation optimisation Tensor-based Tensor network con- traction optimisation Data format optimisation Precomputation optimisation FPGA Schrödinger Computation optimisation Fig. 10. Summary of acceleration methods. highly efficient. Thus, using tensor networks allows for the simulation of only one or a small batch of state amplitudes at the end of the circuit. The complexity of the tensor network is constrained by the largest tensor involved in the contraction process. 5 Simulation Hardware Platforms Based on the size of the problem, various hardware platforms with different computational power can be used to run the simulations. Nevertheless, all of them will eventually be limited by either the required memory or the simulation time. The exponential growth of memory and simulation time is the reason for the relevance of the simulation optimization efforts, which will be discussed later in Sections 6 to 8. A diagram presenting the techniques discussed in this work is depicted in Figure 10.

5.1 Small-scale quantum simulation Currently, an algorithmic-level simulation of 30 qubits with Intel quantum SDK requires up to 34GB of free RAM. Increasing this by just two additional qubits would require up to 135GB. This means that with today's technology, even a common personal computer can be used for a moderately small number of qubits. Personal computers are the most commonly available platform for running simulations, primarily using the CPU and, in some cases, the GPU. Personal computers can also be combined with an external FPGA, which allows off-loading some of the work. As described in Section IV-C, in state- vector simulations, the core operation is matrix multiplication, which, if properly handled due to 14 A. Cicero et al. the use of complex numbers, can be processed inside the FPGA and the results then transferred back to the CPU.

5.2 Medium-scale quantum simulation Scaling is quite an issue in quantum computer simulations, as mentioned earlier. This increase in computational resources can be supported by better simulation hardware, such as workstations. These are typically equipped with state-of-the-art CPUs and one or more GPUs, offering more memory compared to a standard personal computer. This enables the simulation of a larger number of qubits, although still limited due to the exponential scaling of memory requirements. With current technology, simulating up to 32 qubits is possible using commercially available workstations. Another factor to consider is the simulation time. Upgrading to workstations, especially those equipped with multiple GPUs, generally enables faster execution if the simulator is optimized for parallel execution. Additional speedup can be achieved if the workstation has a processor with a matrix unit, which allows for quicker execution of core operations.

5.3 Simulation beyond the medium-scale If the goal is to run simulations with the highest possible number of qubits or at the fastest speed possible, a high-performance computer is required. An example of this is the execution of 61-qubit quantum circuits on Argonne Theta using qHIPSTER, an earlier version of the Intel quantum simulator. The simulation, although optimized, required

0.8 Pb of memory, a level of resources not available on standard laptops or workstations. The advantage of high-performance computers is that they are equipped with multiple nodes, each composed of multiple CPU cores or GPUs, allowing for significantly better performance compared to a single laptop or workstation. 6 Acceleration using CPU Table 1. Summary of most important acceleration works with CPU. Work Simulation type Baseline Improvement Benchmark Platform Tensor-based 120-qubit QAOA 210-qubit QAOA in 64s Theta Supercomputer Tensor-based 81-qubit RQC, 40 depth [22] 100-qubit RQC, 42 depth in 304s Sunway Supercomputer Schrödinger 45 qubits, 26 depth, 0.5PB mem 45 qubits, 27 depth, 17.4GB mem in 1.4h Google Cloud Schrödinger 36 qubits in 194.12s 36 qubits in 94.48s Linux Server Schrödinger 49 qubits, 27 depth in ≥ 24 h 49 qubits, 27 depth in 1.49h Sunway Schrödinger 47 qubits, 2.8PB mem 61-qubit Grover's search, 0.8PB mem Argonne Theta Schrödinger 45 qubits, JUQCS-E 48 qubits in 300min K Computer (RIKEN) CPU simulation acceleration focuses mainly on scalability, and in particular, most of the work reported here tries to optimize the simulation on big clusters of CPUs, with high-performance com- puters such as Summit, Sierra (Lawrence Livermore National Laboratory), or Theta (Argonne National Laboratory).

The optimizations presented are classified by their approach: Schrödinger or Tensor-based. Table 1 summarizes all the results.

6.1 Schrödinger-style simulation acceleration As described previously in Section 4.3, accelerating Schrödinger-style simulations consists of speeding up the core matrix multiplication. Different approaches have been taken to achieve improvements.

6.

1.1 Baseline simulation algorithm. The most mathematically simple way to execute full state- vector simulations for an n -qubits circuit with quantum gates, reported in the work of Fatima and Markov, is to: (1) Order the gates left to right (2) Execute each layer of gates in parallel. All qubits must go through either an actual gate or an identity matrix; therefore pad each gate with an identity matrix of an appropriate dimension via Kronecker products to obtain a $2^n \times 2^n$ matrix (3) Multiply all matrices in order This allows the entire circuit to be represented as a matrix, and multiplying it with the input state vector produces the output. Although it is a simple algorithm, it is not the most efficient method in terms of memory utilization and parallelizing computations. Another straightforward approach is to apply each gate directly to the input state vector. Generally, in most works, the simulations are performed in two main steps. The first step is to partition the circuit into multiple sub-circuits, which can be executed in parallel, eventually grouping or reordering gates within the same partition. The second step involves adopting various strategies to optimize the execution. Additional techniques, such as compression and optimizing the data format, can further enhance performance.

6.

1.2 State-vector compression. One of the main issues of the Schrödinger simulation, mentioned already before, is memory usage. Storing the full state vectors during the simulation requires an exponentially increasing memory space. A possible solution, used already in other applications that handle vast volumes of data, is the use of data compression techniques. There are mainly two types of compressions: lossless compression and error- bounded lossy compression [28, 104, 111–113, 147]. Introducing a module in the simulation workflow that compresses the state vectors before storing them in memory and decompresses them when needed helps address the memory usage problem. Lossless compression is an approach used in works such as Zhao et al.. Their work focuses on reducing the data transfer between CPU and GPU, and one of their contribution is the lossless data compression of non-zero amplitudes, achieved by observing how the state vector after each operation has similar amplitude values. Lossy compression of state vectors is the approach used in works such as Wu et al.. That approach allows to simulate Grover's search algorithm, as well as other quantum algorithms such as Quantum Approximate Optimisation Algorithm (QAOA) and random circuit proposed by Google, with up to 61 qubits. This is achieved by compressing the state vectors and reducing the memory size required from 32 EB to 768 TB. For the data compression, the authors implement an error-bounded lossy compressor, a technique that limits the compression ratio based on the corresponding fidelity loss. It is possible to select the optimal compression strategy during different parts of the simulation, using Zstd alongside an error-bounded lossy compression method developed by the authors. Another approach,, is to use an adaptive encoding scheme based on the polar representation $\alpha = r e^{i\theta}$ of the complex number α for the state-vector elements. One byte is used to encode the angle $-\pi \leq \theta \leq \pi$ and the remaining bytes encode the value r , after scaling based on the maximum and minimum values of the state vector. This allows for a reduction in the amount of memory required to store the state by a factor of 8, but it requires additional time to perform the encoding and decoding procedure, up to a factor of 3-4.

6.

1.3 Data format optimisation. While the 64-bit double representation is the most common approach to the data representation, there have been efforts to reduce the number of bits 16 A. Cicero et al. required to obtain results with a good enough fidelity. This allows for a lower memory footprint and higher throughput. The work of Fatima and Markov uses the 32-bit float type representation instead of the more common 64-bit

double type, therefore reducing the total amount of necessary resources. The approximation error is controlled by globally keeping track of the changes in amplitude for changes bigger than 1. This is done because most of the gates do not significantly change the amplitude value. Gates more commonly act on the phase, so the result is not at risk of underflow. An underflow occurs when the result of an arithmetic operation is relatively so small that it can not be stored in the input operand format without resulting in a rounding error that is larger than usual. But some gates, such as the Hadamard gates, can change the amplitude by a factor of 1 or $\sqrt{1}$, generating underflow issues with the 32-bit float representation. In this case, the change is tracked and included at the end of the quantum circuit, allowing the reduction of memory needed while still obtaining results with good fidelity. The same research group published a follow-up work where their simulation acceleration techniques are optimized in terms of parallelization, with a focus on allowing the simulation to be run on generic hardware.

6.

1.4 Gate clustering and circuit partitioning. Simulating gates one at a time, as described in the baseline algorithm, is slow because it requires separate memory traversals. A common technique to avoid this issue is to simulate gate clusters in batches. A gate cluster is obtained by combining the matrices representing the gate matrices acting on one or multiple qubits. The common approach is to cluster adjacent gates acting on the same qubit. Google QSim merges each one-qubit gate to some nearby two-qubit gate. Another work applies gate clustering to a larger amount of qubits, up to five, and then multiplies out the obtained gate matrices. The work of Fatima and Markov creates clusters of $\log(n)$ qubits out of the total amount of n qubits that grows as $\log(n)^2$. They apply gate clustering by reordering, a technique that loops over the circuit and clusters adjacent gates of the same type or reorders non-adjacent gates to form larger clusters, allowing for optimized algorithms for each type of cluster. The approach of qTask is to partition a state vector into a set of blocks, with each partition spawning one or multiple tasks performing gate operations on designated memory regions. This, as the previous techniques, allows to enable inter-gate operation parallelism due to the breaking down of the gate dependencies. Another strategy presented in is the technique named implicit decomposition. The target circuit is efficiently partitioned into different parts with a focus on balancing the memory requirements for each one of them. This efficient partitioning allows to save memory space compared to storing the entire state vector, because it is not necessary to have all the amplitudes after the individual calculations. They additionally propose a dynamic algorithm to select the optimal partition scheme.

6.

1.5 Memory access optimization. Given that the execution is dominated by matrix multiplication operations, an obvious optimization is to do them in a cache-friendly way, resulting in improved performance. Memory locality can be exploited with gate clustering. If paired-up gates act on qubits as closely as possible, it reduces memory strides (distance between two successive elements of an array in memory) when simulating gate pairs acting on less significant bits. Another technique, is cache blocking, where rather than applying pairs of qubit gates in separate passes, such pairs, acting on different qubits, are reordered and applied partially in different orders. Each state vector is divided into chunks, which fit in the L2 cache, and for each chunk, multiple non-overlapping pairs of one-qubit gates and an occasional unpaired gate are applied to it. This reduces the cache misses and improves the performance. The work of Fang et al. tackles the memory locality with their hierarchical simulation framework. The authors consider that scaling up means the working set size of the simulation set would exceed the cache size of a modern CPU. To address this, the input circuit is executed as a sequence of sub-circuits, each containing a portion of the original gates, allowing for better locality.

6.

1.6 Parallel execution. Efficiently handling the parallel execution of different gates or tasks is crucial to capitalizing on the advantages introduced by the circuit modifications mentioned earlier. Most works utilize a

multi-CPU approach with OpenMP and MPI-based CPU simulations. The work of Fatima and Markov explains in detail how to exploit CPU architecture to simulate different clusters, exploring data-level parallelism.

6.2 Tensor-based simulation acceleration Tensor network-based simulations are divided into two main steps. The first one is to represent the quantum circuit as a tensor network. The second step is to apply the contraction to the tensors, as described previously in Section 2.4. We now present the different acceleration solutions for each step of the simulation.

6.

2.1 Tensor network creation. For the first step of the simulation, different approaches can be taken depending on the type of algorithm and the structure of the quantum circuit. In the case of Random Quantum Circuits (RQC), a viable solution is using Projected Entangled Pair States (PEPS) representation of quantum states from many-body quantum physics. This representation, according to works for 2×2 by 2×2 lattice types, but to work with different structures, the generation of the best contraction path (choice of which nodes to contract at different steps of the simulation) becomes a bigger issue. A possible approach is to use the CoTenGra software to look for the best path. It uses a loss function that combines the considerations for both the computational complexity and the compute density, which are determinant factors for the performance on a many- core processor. Another approach is to use ordering algorithms. There are several available ordering algorithms, classified as:

- Greedy algorithm: usually used as a baseline, contracts the lowest-degree vertex in a graph.
- Randomized greedy algorithms: The computational cost of the contraction at each step is function of the maximum number of neighbours for a node. Minimizing this by choosing the optimal contraction order allows for a decreased computation cost. This approach can do so without prolonging the run time.
- Heuristic solvers: Attempt to use some global information in the ordering problems. Example of this are QuickBB and Tamaki's heuristic solver. Table 2. Summary of most important works acceleration performances with GPU. Work Simulation type Baseline Improvement Benchmark Platform Average contraction speedup: 4.05× to, NVIDIA H100 GPU, [11] Multiple 53-qubit RQC, depth 10 NVIDIA A100 GPU, 4× to, 547.35× to AMD EPYC 7742 CPU Tensor based 42 qubits in 1965.52s 42 qubits in 195.71s NVIDIA A100 GPU Tensor based 30 qubits, depth 4 in 246s 30 qubits, depth 4 in 1.4s GPU Schrödinger 30-37 qubits circuits in range 10s to 100s 15.8% average runtime reduction NVIDIA V100 GPU Schrödinger 35-qubit QFT 35-qubit QFT, 80% speedup 6× NVIDIA V100 GPU Schrödinger 26 qubits in less than 10ms 26 qubits, ≥ 10× speedup GPU Cluster Schrödinger 22-qubit QFT Noisy QFT, 22 qubits, up to 10× speedup GPU 18 A. Cicero et al.

6.

2.2 Tensor-network contraction. Depending on the optimisations done during the tensor creation step, the contraction can be more or less efficient. During the contraction phase, an optimal slicing method is required to divide the tensor network into different clusters. This allows an efficient parallelization of the computations, to efficiently process all the sub-tasks of the circuit across all the available nodes. This is necessary to balance the compute and storage costs. In the graph representation, the contraction of the full expression is done by consecutive elimination of graph vertices. The consequence is that the vertex is removed from the graph and the neighbours are connected. The slicing of a tensor over an index means evaluating many variables while keeping one of them constant. Finding the optimal index to slice is the focus of this process. An approach to this is to select the best index after each step of the slicing, a method known as step-dependent slicing. Another approach is to divide the contraction into two steps, the index permutation of the tensors as a preparatory step and the second step is the following matrix multiplication to achieve the contraction results. Permutation of indices is generally required to convert the tensor contractions into efficient matrix multiplications. In the case of tensor contraction on many- core processors with a high

compute density of high-rank tensors, it is important to reduce the permutation cost to reduce the movement of data items with strides in between, which is unfriendly for current memory systems. A proposed approach is to use fused permutation and multiplication, which means using different compute processing elements in a collaborative way, where each CPU reads its corresponding data block in a regular, constant stride, pattern, thus achieving a high utilization of the memory bandwidth. Parallelization of the tensor computation is done after the slicing. A possible approach is to use a two-level parallelization architecture. On a multi-node level, the partially contracted full expression is sliced over $\square$ indices and distributed to $\square \times 2$ message-passing interface ranks. Node-level parallelism over CPU cores is done using system threads. For every tensor multiplication and summation, the input and output tensors are sliced over $\square$ indices. The contraction is then performed by $2 \times \square$ threads writing results to a shared result tensor.

6.

2.3 Data format optimisation. To achieve a high accuracy in the computation of the simulated output state compared to the actual output state it is necessary to represent the data with a correct amount of bits. The default approach is to use 64-bit double (floating-point) representation. In the case of RQCs, the work of Yong et al. proposes an adaptive precision scaling, which adjusts the data precision to single or half-precision dynamically depending on the degree of the sensitivity of different parts of the computation.

6.3 Other acceleration approaches Alternatively to what was described previously, some works exploit other approaches for optimizations, such as Quantum Processing Units (QPUs) and different memory technologies. For example, CutQC employ classical CPUs interacting with a small quantum computer. This hybrid computing approach enables the evaluation of larger quantum circuits that cannot be executed on a classical computer alone. It offers better fidelity and allows the simulation of larger circuits than either approach could achieve independently. SnuQS focuses on the full utilization of the storage device connected to the system. That work aims to achieve maximum I/O bandwidth by employing memory management and optimization techniques, resulting in better performance compared to DDR4 DRAM main-memory-only systems. Other approaches that make use of multiple hardware platforms can be found in the literature [7](#)

Acceleration using GPU Accelerating quantum computer simulations using GPUs has been an effective approach for many years. Early works, such as the one by Gutierrez et al., attempted this over 10 years ago. Similar to CPU simulations, GPU simulations can also be classified according to the classification defined in Section 4. The results from the most relevant works are presented in Table 2.

7.1 Schrödinger-style simulation acceleration As seen in Section 6, the core operations of quantum circuit simulation are small matrix multiplications. If the data dependencies between these operations are limited or nonexistent, and they can be parallelized, GPUs can perform them with much higher throughput compared to CPUs. It is still important, even for GPU execution, to focus on data locality to minimize memory access misses. In GPU simulation acceleration, each work focuses on one or both of two steps: first, partitioning the circuit into sub-circuits or tasks; second, accelerating the computation.

7.

1.1 Circuit Partitioning. As seen for the CPU in Section 6, also in the GPU approach the first step followed is to partition the circuit in order to enable a more efficient parallel execution. HyQuas proposes a circuit-aware partition strategy and a high-accuracy performance model that guides the partitioning. This allows to obtain a near-optimal partition of a given quantum circuit into different groups and select an optimal method to compute each one. CuStateVec, a library of the cuQuantum SDK [\[11\]](#), which focuses on state-vector simulation acceleration for GPU, uses gate fusion. Numerous small gate matrices are fused into a single multi-qubit gate matrix, which can then be computed in one shot instead of performing multiple computations. This allows for improved performance in cases where both the high compute performance and high memory bandwidth of the GPU are used.

7.

1.2 Memory access optimisation. Different approaches can be used to address the data locality issue. ShareMem method considers a circuit partitioned in several gate groups, which are applied only to a subset of n qubits out of the N total qubits. The total 2^n state vector values can be n split into fragments of size 2^k , which can be stored in GPU shared memory, and each fragment can be mapped to one GPU thread block. The thread block can load the fragment from the global memory to the shared memory, apply the gates on it, and store back the fragment in the global memory. This method performs better than the following BatchMV approach in the case of a sparse part of the circuit. The BatchMV method is based on the idea that if all the target and control qubits of a gate group come from the same n qubits, it is possible to merge these gates into a n -qubit gate. This allows to divide the simulation into 2^{n-k} matrix-vector multiplication tasks, each one with a $2^k \times 2^k$ gate matrix to a state vector of 2^n values. The index of the values only differs on these k positions, improving the data locality. This method performs better than the previous in the dense part of the circuit. HyQuas automatically selects the best approach depending on the part of the circuit between OShareMem, an improved version of ShareMem and TransMM, which performs better than BatchMV in their framework. TransMM transposes the quantum state, allowing to treat the gate-applying operation into a standard GEneral Matrix Multiplication (GEMM) operation, which can be accelerated by highly optimized libraries such as cuBLAS and Tensor Cores.

7.

1.3 Parallel execution. As seen in Section 6, the main way to approach distributed computing is with OpenMP or MPI. It is important to maximize GPU usage while minimizing data exchange between the CPU and GPU, as frequent amplitude exchanges between them introduce significant data movement and synchronization overhead. Qubit reordering is also a technique used by cuStateVec [11] to address the challenge of applying a gate onto a global indexed qubit. The 20 A. Cicero et al. approach involves moving the qubit from a global index to a local index, allowing a single GPU to compute the state vector. On a more broad approach, there are even works proposing a novel multi-GPU programming methodology, such as the work from Li et al., which constructs a virtual BSP machine on top of modern multi-GPU platforms.

7.2 Tensor-based simulation acceleration As reported in the work of Vincent et al. a major issue in quantum computer simulations is the under-utilization of GPUs and CPUs in supercomputers. They further predict an increase in inefficiencies due to the increase of parallelism and heterogeneity in exascale computers, with billions of threads running concurrently. Future supercomputers will see an increase in on-node concurrency rather than the number of nodes, with large multi-core CPUs and multiple GPUs per node.

7.

2.1 Tensor network creation. For this step of the simulation on GPUs, some works also use CoTenGra, as already mentioned in Section 6, to compute high-quality paths and slices for tensor networks. Additionally, there is the possibility of reusing common calculations, as described in Ref., which avoids adding duplicate tasks during the slicing, therefore leading to a task graph with shared nodes.

7.

2.2 Tensor network contraction. A possible approach, presented for Jet is to use the transpose-transpose-matrix-multiply method. The basic building block of the task-dependency graph is the pairwise contraction of two tensors. This can be decomposed into two independent (partial) tensor transposes and a single matrix multiplication. While for the CPU they use the qFlex [6] transpose method, for the GPU they use cuTENSOR [11], used also in the work of Shah et al.. Another approach to tensor contraction is the bucket contraction algorithm by Dechter, as described in QTensor and used also by Shah et al., where an ordered list of tensor buckets (collection of tensors) is created to contract the tensor network. Each bucket corresponds to a tensor index, the bucket index. Buckets are then contracted one by one. The contraction of a bucket is performed by summing over the bucket index, and the resulting tensor is then appended to the appropriate bucket. The number of unique indices in aggregate indices of all bucket tensors is called a bucket width. Memory and computational resources of a bucket contraction scale exponentially

with the associated bucket width. It is possible to improve this method by ordering the buckets first and then finding the indices that can be merged before performing the contraction. This allows for a smaller output size and larger arithmetic intensity as presented by Lykov et al.. An issue with the bucket elimination algorithm is that tensors can grow too large to fit in memory, so a solution proposed by Shah et al. is to introduce data compression. Their work proposes a GPU-based lossy compression framework that can compress floating-point data stored in quantum circuit tensors with optimized speed while keeping the simulation result within a reasonable error bound after decompression. The compressed data can be decompressed when the tensors are needed during the computation. An additional approach is to use a partitioning method, which is optimal for each part of the circuit, depending on if it is sparse or dense, as mentioned already in the Schrödinger approach in the same work of Zhang et al.. OShareMem method, an optimized version of the ShareMem method, deletes redundant computation, reduces data indexing overhead, and uses a new layout to access the shared memory faster. Another approach is the TransMM method, which converts a set of special matrix-vector multiplications in quantum circuit simulation into GEMM to take advantage of highly optimized GEMM libraries and hardware-level GEMM compute units like Tensor Cores. The work of Zhang et al., as mentioned before, introduces an automatic selection method for the approach to be used, by using OShareMem for sparse parts of the circuit, and TransMM method for the dense parts of the circuit, allowing for further speedup.

7.

2.3 Data format optimisation. As already mentioned for the CPU optimizations, using smaller precision for the data and calculations leads to improved performance. Although the works that try to accelerate the tensor-based acceleration on GPU do not try to propose any optimisation regarding the data format, they pick slightly different configurations. Most of the works use double (64-bit) floating-point precision, while other works, such as Jet, use different precision such as single (32-bit) floating-point precision.

7.3 Other acceleration approaches Some simulation tools allow for the acceleration of quantum circuit simulation on GPU by focusing more on the optimal usage of the available hardware. An example of this is cuQuantum [11], which provides composable primitives for GPU-accelerated quantum circuit simulation, including distributed computing on multiple GPUs. TensorLy-Quantum is a quantum library with direct support for tensor decomposition, regression, and algebra. It additionally provides built-in support for Multi-Basis Encoding for MaxCut problems and was used to develop Markov chain Monte Carlo-based variational quantum algorithms. 8 Acceleration using FPGA Although FPGAs are more limited in terms of resources and frequency when compared to CPUs and GPUs, they allow for more flexibility, leading to architectures better fitted to problem. While less scalable compared to the implementations with the previous hardware platforms, FPGAs excel when working with very limited resources, such as simulations running on an off-the-shelf personal computer. It is worth noting that some of the reported work, especially early efforts, focused on emulating quantum hardware rather than serving as an acceleration platform for quantum computer simulation, where only a few core operations are offloaded to the FPGA. In the former case, the FPGA acts as a quantum computer itself, with which the host system must interface. Some example of this emulation approach include the early works of Khalid et al. and Aminian et al. [5]. This same approach has been followed more recently in terms of work of Zhang et al.. These approaches are interesting but can only be applied for problems requiring only a very limited number of qubits. More recent works focus on running only the most compute-heavy operations on the FPGA. FPGA-accelerated simulation mainly focuses on the previously discussed Schrödinger approach. The two main steps of the acceleration are the pre-computation of the circuit matrix and the matrix computation.

8.1 Pre-computation optimisation In the pre-computation step, various optimisations have been proposed. The work of Jungjarassub and Piromsopa introduces an algorithm that does not directly multiply all the gates, but first checks for special cases. Depending on the combination of gate types and the transformation they apply, they can avoid executing certain multiplications in certain specific cases. A similar approach is used in the work of Hong et al., which checks for the matrix values before multiplication, and in the case

where ones or zeros are present, avoids processing through the actual multiplication module. This work focuses on the computation of a 2×2 gate regardless of the circuit. It first groups multiple gates into a single one when possible, then simulates the 22 A. Cicero et al. resulting gates one by one. Although this approach does not prioritize parallel execution, it allows for the execution of larger circuits on hardware with limited resources. Table 3. Hardware-aware optimization techniques. Technique Improves Hardware support Mixed-precision operations compute & data storage CPU, GPU Arbitrary-precision operations compute & data storage FPGA Lossy data compression data storage CPU (software), GPU (software), FPGA (compression engine) [24] Thread-/Task-level parallelism compute CPU, GPU Vector (SIMD) instructions compute CPU with vector support Matrix operations compute CPU with MMU, GPU tensor cores, FPGA Systolic Array, AI engines

8.2 Computation optimisation Different approaches have been proposed to speed up the actual calculation. In the case of gates that generate phase rotation, it is possible to use a look-up table to collect sine and cosine values instead of calculating them directly. Look-up tables are also evaluated in the work of Mahmud and El-Araby, which replaces any complex calculations with a simpler array-indexed operation. This is further optimized by storing only the value relative to the actual input vectors which are necessary for the simulation. Another optimization to save memory operations is to check whether the state vector changes after a gate is applied before saving it back to memory. If no changes are detected, the memory-write can be avoided as is done in the work of Hong et al.. The work of Mahmud and El-Araby evaluates different hardware architectures to identify the best-performing one for the Complex Multiply and ACcumulate (CMAC) operation, which is the core of simulation. They evaluate a single CMAC unit which processes all the operations (optimized for area but with low throughput), N-concurrent-CMAC units (optimized for throughput but requires a higher number of CMAC instances) and a dual-sequential-CMAC architecture (two CMAC instances connected sequentially, with computation and data write operations overlapped). Lastly, they also propose a kernel-based emulation model useful in case of a repeated set of core operations. The follow-up work by Mahmud et al. proposes an additional approach that improves the scalability of the emulation. Instead of using a look-up table, which sacrifices area for speed, or using dynamic generation (generation of the algorithm matrix elements at compile-time), which sacrifices speed for area, they propose a stream-based CMAC. They stream the algorithm matrix elements at run-time, meaning the operation's cost is typically the I/O channel latency between the control processor and the FPGA, which is negligible compared to the time required for processing the algorithm matrix.

9 Summary and Future Directions In this work, we examined different approaches to speeding up and reducing the memory usage for the simulation of quantum computers. The most important results, reported in Table 1 for the CPU-related works and in Table 2 for the GPU-related works, show how this is possible with multiple different approaches. Focusing on the hardware-aware optimizations for the simulation problem and challenges, we present in Table 3 a summary of the different techniques, what they aim at improving, and which hardware support is best for that purpose. For each hardware support we also include a reference to a sample work showing a representative implementation of the technique in domains other than quantum computing. The first group of optimizations aim at improving both the computation and data storage by changing the data and operations to use reduced precision. This has obvious benefits, but the challenge is to control the error introduced, since it is an optimization that loses information from the original 64-bit double floating-point precision. For these optimizations, FPGAs can offer the benefit to implement hardware that can operate on data of arbitrary precision (not necessarily conforming to standard sizes). Using reduced-precision data and operations is an optimization that is successfully being exploited in the Machine Learning (ML) domain currently by applying quantization of the data values. The second group of optimizations are those focusing on reducing the data storage and are based on data compression techniques. In this case, since the goal is to reduce the data size considerably, lossless techniques are not enough and thus a lossy technique needs to be explored. This technique again comes with the price of reduced accuracy since there is a reduction of information after compressing the data. Several techniques have been applied in the past for scientific applications [19] and in order for the techniques to be applicable with reduced latency, it is important to have a hardware

compression module [24]. The third group of optimizations focus on lowering the execution time by leveraging parallel processing and/or using dedicated hardware units for some demanding operations. Thread and task parallelism is an effective technique to reduce the execution time by exploiting the existing parallel hardware resources in CPUs and GPUs. Single-Instruction-Multiple-Data (SIMD) instructions are used to exploit parallelism at a finer-grained scale, at the instruction level. When using SIMD instructions we are using multiple computational units to perform multiple operations in the same cycle. SIMD support is now common in most commodity processors, but the other operations that are very relevant to these simulations are matrix operations, which usually require dedicated matrix units for acceleration. Since matrix operations are also very relevant for AI, there has been a recent increase in products providing hardware support for these operations. Namely, some CPUs and GPUs now have dedicated matrix units and/or neural acceleration units. Also, the Google TPU used for both ML training and inference is basically a hardware accelerator for the matrix-matrix operations. The use of FPGAs in this case is also very relevant as they can be used to implement matrix-matrix operations for certain non-standard matrix dimensions. Also, as mentioned previously, FPGAs could be exploited to directly implement matrix-matrix operations for complex numbers. Considering all of the above, we believe that in the future there is a need to invest in the development of more effective data compression techniques, a more flexible use of data precision. Since the market share for the simulation of quantum computers is much smaller than for AI applications, we do not expect an explosion of accelerators as has been observed in the recent years for the ML domain. As such, it is unlikely that we will see many hardware accelerators dedicated to the simulation of quantum computers. But the developments in the ML domain can be leveraged to deliver benefits to this domain too, since the critical operations are basically the same: matrix-matrix operations. As such, we expect in the future to see the use of ML accelerators for improving the performance of quantum computer simulations. Lastly, it is very interesting to already see some development using FPGAs and we expect the trend will be to see more and more FPGA-based dedicated hardware to solve specific operations on specific data types in a very efficient way.

10 Conclusions

Quantum computer simulators play an extremely important role in helping the development of new algorithms and hardware for the promising quantum computing paradigm. While several approaches and simulators have been proposed, the characteristics of the problem make it extremely hard to scale to systems with a larger number of resources, required for solving more complex problems. The main challenges of quantum computer simulations are the large amount of data that must be stored in memory, with sufficient precision to produce results with acceptable accuracy, and the execution time, which grows exponentially with the increased number of qubits. In this work, we presented a review of existing tools and approaches for systems with CPUs, GPUs, and FPGAs, with a focus on how hardware-aware optimizations can help address the challenges. Based on this study we showed the future directions for hardware-aware optimizations, including the use of accelerators designed for other domains but addressing similar problems and the development of FPGA-based hardware accelerators for quantum computer simulation.

Acknowledgments

We acknowledge support from the Swedish Foundation for Strategic Research (grant number FUS21-0063), the Horizon Europe programme HORIZON-CL4-2022-QUANTUM-01-SGA via the project 101113946 OpenSuperQPlus100, and from the Knut and Alice Wallenberg Foundation through the Wallenberg Centre for Quantum Technology (WACQT). AFK is also supported by the Swedish Research Council (grant number 2019-03696) and the Swedish Foundation for Strategic Research (grant number FFL21-0279). The work is also partially funded by the European High Performance Computing Joint Undertaking (JU) under Framework Partnership Agreement No 800928 and Specific Grant Agreement No 101036168 (EPI SGA2). The JU receives support from the European Union's Horizon 2020 research and innovation programme and from Croatia, France, Germany, Greece, Italy, Netherlands, Portugal, Spain, Sweden, and Switzerland.

References

- [1] Scott Aaronson and Lijie Chen. 2017. Complexity-theoretic foundations of quantum supremacy experiments. In Proceedings of the 32nd Computational Complexity Conference (Riga, Latvia) (CCC '17). Schloss

Dagstuhl–Leibniz- Zentrum fuer Informatik, Dagstuhl, DEU, Article 22, 67 pages.

[2] COMSOL AB. 2024. COMSOL Multiphysics. <https://www.comsol.com/>

[3] Tahereh Abad, Jorge Fernández-Pendás, Anton Frisk Kockum, and Göran Johansson. 2022. Universal Fidelity Reduction of Quantum Operations from Weak Dissipation. *Physical Review Letters* 129 (Oct 2022), 150504. Issue 15. <https://doi.org/10.1103/PhysRevLett.129.150504>

[4] Rajeev Acharya et al. 2024. Quantum error correction below the surface code threshold. *arXiv:2408.13687*

[5] Mahdi Aminian, Mehdi Saeedi, Morteza Saheb Zamani, and Mehdi Sedighi. 2008. FPGA-based circuit model emulation of quantum algorithms. In *2008 IEEE Computer Society Annual Symposium on VLSI*. IEEE, 399–404.

[6] Frank Arute et al. 2019. Quantum supremacy using a programmable superconducting processor. *Nature* 574 (2019), 505. <https://doi.org/10.1038/s41586-019-1666-5>

[7] Jose Aumentado. 2020. Superconducting Parametric Amplifiers: The State of the Art in Josephson Parametric Amplifiers. *IEEE Microwave Magazine* 21, 8 (2020), 45–59. <https://doi.org/10.1109/MMM.2020.2993476>

[8] Medina Bandic, Sebastian Feld, and Carmen G Almudever. 2022. Full-stack quantum computing systems in the NISQ era: algorithm-driven and hardware-aware compilation techniques. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.

[9] Mari Carmen Bañuls. 2023. Tensor network algorithms: A route map. *Annual Review of Condensed Matter Physics* 14, 1 (2023), 173–191.

[10] BelaBauer, SergeyBravyi, MarioMotta, and Garnet Kin-LicChan. 2020. QuantumAlgorithmsforQuantumChemistry and Quantum Materials Science. *Chemical Reviews* 120, 22 (2020), 12685–12717. <https://doi.org/10.1021/acs.chemrev.9b00829> *arXiv:https://doi.org/10.1021/acs.chemrev.9b00829*

[11] Harun Bayraktar, Ali Charara, David Clark, Saul Cohen, Timothy Costa, Yao-Lung L Fang, Yang Gao, Jack Guan, John Gunnels, Azzam Haidar, et al. 2023. cuQuantum SDK: A high-performance library for accelerating quantum science. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1. IEEE, 1050–1061.

[12] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, Shah Nawaz Ahmed, Vishnu Ajith, M Sohaib Alam, Guillermo Alonso-Linaje, B AkashNarayanan, Ali Asadi, et al. 2018. PennyLane: Automatic differentiation of hybrid quantum-classical computations. (2018). *arXiv:1811.04968*

[13] Ethan Bernstein and Umesh Vazirani. 1997. Quantum Complexity Theory. *SIAM J. Comput.* 26, 5 (1997), 1411–1473. <https://doi.org/10.1137/S0097539796300921> *arXiv:https://doi.org/10.1137/S0097539796300921*

[14] Kishor Bharti, Alba Cervera-Lierta, Thi Ha Kyaw, Tobias Haug, Sumner Alperin-Lea, Abhinav Anand, Matthias Degroote, Hermann Heimonen, Jakob S. Kottmann, Tim Menke, Wai-Keong Mok, Sukin Sim, Leong-Chuan Kwek, and Alán Aspuru-Guzik. 2022. Noisy intermediate-scale quantum algorithms. *Reviews of Modern Physics* 94 (Feb 2022), 015004. Issue 1. <https://doi.org/10.1103/RevModPhys.94.015004>

[15] Dolev Bluvstein, Simon J. Evered, Alexandra A. Geim, Sophie H. Li, Hengyun Zhou, Tom Manovitz, Sepehr Ebadi, Madelyn Cain, Marcin Kalinowski, Dominik Hangleiter, J. Pablo Bonilla Ataides, Nishad Maskara, Iris Cong, Xun Gao, Pedro Sales Rodriguez, Thomas Karolyshyn, Giulia Semeghini, Michael J. Gullans, Markus Greiner, Vladan Vuletić, and Mikhail D. Lukin. 2024. Logical quantum processor based on reconfigurable atom arrays. *Nature* 626 (2024), 58–65. <https://doi.org/10.1038/s41586-023-06927-3>

[16] Sergio Boixo, Sergei V. Isakov, Vadim N. Smelyanskiy, Ryan Babbush, Nan Ding, Zhang Jiang, Michael J. Bremner, John M. Martinis, and Hartmut Neven. 2018. Characterizing quantum supremacy in near-term devices. *Nature Physics* 14, 6 (April 2018), 595–600. <https://doi.org/10.1038/s41567-018-0124-x>

[17] Jonathan J. Burnett, Andreas Bengtsson, Marco Scigliuzzo, David Niepce, Marina Kudra, Per Delsing, and Jonas Bylander. 2019. Decoherence benchmarking of superconducting qubits. *npj Quantum Information* 5, 1 (26 Jun 2019), 54. <https://doi.org/10.1038/s41534-019-0168-5>

[18] Jonathan J Burnett, Andreas Bengtsson, Marco Scigliuzzo, David Niepce, Marina Kudra, Per Delsing, and Jonas Bylander. 2019. Decoherence benchmarking of superconducting qubits. *npj Quantum Information* 5, 1 (2019), 54.

- [19] Franck Cappello, Sheng Di, Sihuan Li, Xin Liang, Ali Murat Gok, Dingwen Tao, Chun Hong Yoon, Xin-Chuan Wu, Yuri Alexeev, and Frederic T Chong. 2019. Use cases of lossy compression for floating-point data in scientific data sets. *The International Journal of High Performance Computing Applications* 33, 6 (2019), 1201–1220. <https://doi.org/10.1177/1094342019853336> arXiv:<https://doi.org/10.1177/1094342019853336>
- [20] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. 2021. Variational quantum algorithms. *Nature Reviews Physics* 3, 9 (01 Sep 2021), 625–644. <https://doi.org/10.1038/s42254-021-00348-9>
- [21] M. Cerezo, Guillaume Verdon, Hsin-Yuan Huang, Lukasz Cincio, and Patrick J. Coles. 2022. Challenges and opportunities in quantum machine learning. *Nature Computational Science* 2, 9 (01 Sep 2022), 567–576. <https://doi.org/10.1038/s43588-022-00311-3>
- [22] Jianxin Chen, Fang Zhang, Cupjin Huang, Michael Newman, and Yaoyun Shi. 2018. Classical simulation of intermediate-size quantum circuits. (2018). arXiv:1805.01450
- [23] Liangyu Chen, Hang-Xi Li, Yong Lu, Christopher W. Warren, Christian J. Križan, Sandoko Kosen, Marcus Rommel, Shahnawaz Ahmed, Amr Osman, Janka Biznářová, Anita Fadavi Roudsari, Benjamin Lienhard, Marco Caputo, Kestutis Grigoras, Leif Grönberg, Joonas Govenius, Anton Frisk Kockum, Per Delsing, Jonas Bylander, and Giovanna Tancredi. 2023. Transmon qubit readout fidelity at the threshold for quantum error correction without a quantum-limited amplifier. *npj Quantum Information* 9, 1 (16 Mar 2023), 26. <https://doi.org/10.1038/s41534-023-00689-6>
- [24] Shifu Chen, Yaru Chen, Zhouyang Wang, Wenjian Qin, Jing Zhang, Heera Nand, Jishuai Zhang, Jun Li, Xiaoni Zhang, Xiaoming Liang, et al. 2023. Efficient sequencing data compression and FPGA acceleration based on a two-step framework. *Frontiers in Genetics* 14 (2023), 1260531.
- [25] Yanhao Chen, Yuwei Jin, Fei Hua, Ari Hayes, Ang Li, Yunong Shi, and Eddy Z. Zhang. 2023. A Pulse Generation Framework with Augmented Program-aware Basis Gates and Criticality Analysis. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 773–786. <https://doi.org/10.1109/HPCA56546.2023.10070990>
- [26] Zhao-Yun Chen, Qi Zhou, Cheng Xue, Xia Yang, Guang-Can Guo, and Guo-Ping Guo. 2018. 64-qubit quantum circuit simulation. *Science Bulletin* 63, 15 (Aug. 2018), 964–971. <https://doi.org/10.1016/j.scib.2018.06.007>
- [27] Sai Pavan Chitta, Tianpu Zhao, Ziwen Huang, Ian Mondragon-Shem, and Jens Koch. 2022. Computer-aided quantization and numerical analysis of superconducting circuits. *New Journal of Physics* 24, 10 (2022), 103020.
- [28] John Clyne, Pablo Mininni, Alan Norton, and Mark Rast. 2007. Interactive desktop analysis of high resolution simulations: application to turbulent plume dynamics and current sheet formation. *New Journal of Physics* 9, 8 (aug 2007), 301. <https://doi.org/10.1088/1367-2630/9/8/301>
- [29] SolidWorks Corp. 2005. Solidworks.
- [30] Alexander Cowtan, Silas Dilkes, Ross Duncan, Alexandre Krajenbrink, Will Simmons, and Seyon Sivarajah. 2019. On the qubit routing problem. In *14th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2019)*, W. van Dam and L. Mancinska (Eds.), Vol. 135. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 5:1—5:32. <https://doi.org/10.4230/LIPIcs.TQC.2019.5>